
Inside JFreeReport

Written by D.M.Gilbert, 6 June 2000.

© 2000 by Simba Management Limited.

*Everyone is permitted to copy and distribute verbatim copies
of this document, but changing it is not allowed.*

Introduction

This document provides detailed information about **JFreeReport** (version 0.50), and is available for download from <http://www.jrefinery.com>. This is the first public release of this document.

What is JFreeReport?

JFreeReport is a Java class library for printing reports. It is written entirely in Java, and should run on any implementation of the Java 2 Platform (JDK 1.2.2 or later). **JFreeReport** obtains data via Swing's `TableModel` interface, and makes use of the Java 2D API for previewing and printing reports.

*Note that this version of **JFreeReport** is a prototype only. You should not use it for production applications – it has not been tested, it will not handle large reports, and you can expect the API to change in future versions.*

Open Source Licence

JFreeReport is licensed using the GNU Lesser General Public Licence (LGPL). The licence appears in Appendix B.

Open source software licences, such as the LGPL used for **JFreeReport**, grant you far more freedom than traditional software licences. However, the licence still places certain obligations on you – please read and understand the terms of the licence before you use **JFreeReport** in your own programs.

If you are not familiar with the concept of open-source software, you can find out more at the following web-sites:

Address:	Description:
http://www.fsf.org	The Free Software Foundation;
http://www.open-source.org	The Open Source Initiative;

Latest Version

This document has been written for version 0.50 of **JFreeReport**. The latest version, plus any bug-reports or other announcements, can be found on my web-site:

<http://www.jrefinery.com>

There are other open-source class libraries available for download from this site, including **JFinance** and **JFreeChart**.

Acknowledgements

I wish to acknowledge and thank the following people for their contribution towards the development of **JFreeReport**:

- Terence Parr, author of ANTLR. ANTLR is a free parser-generator written in Java. **JFreeReport** contains a parser that was developed using ANTLR 2.7.0. You can download ANTLR from <http://www.antlr.org>;
- Dave Postill, who answered a question I posted in a Java newsgroup and supplied code (to find a file on the classpath) that I have adapted and incorporated into **JFreeReport**;

Comments and Suggestions

If you have any comments or suggestions regarding **JFreeReport**, or this document, please send me an e-mail: david.gilbert@bigfoot.com

Installing JFreeReport

You've downloaded the file `jfreereport0-50.zip` to your system – so what next? Read on for an explanation of how to:

- extract the source, documentation and runtime files from the ZIP archive using standard JDK tools;
- run the demo program;
- compile the source;
- re-generate the JavaDoc documentation;

... using the standard JDK tools.

Extracting the Files

The first step is to unpack the files contained in the ZIP archive. We'll use the **jar** tool, since Java's *jar files* are really just ZIP files in disguise. In the directory where you have the `jfreereport0-50.zip` file, type the following command:

```
jar -xvf jfreereport0-50.zip
```

This will extract all the files from the archive. All the files will appear on your disk saved in a folder called `jfreereport`. This folder contains two files and three subfolders, described in the following table:

File/Folder	Description:
jars	A folder containing the JFreeReport run-time jar files;
javadoc	A folder containing the JavaDoc-generated documentation for JFreeReport;
source	A folder containing the source code files;
licence-LGPL.txt	The text of the GNU Lesser General Public Licence;
readme.txt	Information about this version of JFreeReport;

This is everything you need to use **JFreeReport**, apart from this document (which can be downloaded separately).

Running the Demo Program

JFreeReport ships with a simple demo program, to illustrate how reports can be generated using the class library. The `jars` folder (see the previous section) contains the four files (three jars and the `reports.zip` file) that you require to run the demo using the **java** tool. Change to that directory and enter the following command (all one line) to run the demo:

```
java -cp jfreereport.jar;demo.jar;antlr.jar  
com.jrefinery.report.demo.JFreeReportDemo
```

You don't need to worry about compiling the source—the jar files contain precompiled class files.

Compiling the Source

You can compile the source files (contained in the `source` folder) using the **javac** tool, although I'd recommend setting up a project in your favourite development environment and working from there. But, just to prove that it can be done...

Change directory to the `source` folder, then type the following command (again, all one line):

```
javac -g:none -O -verbose -classpath ../jars/antlr.jar  
com/jrefinery/report/demo/JFreeReportDemo.java
```

This compiles the demo application *and all the JFreeReport class library files* (**javac** compiles each class for which it cannot find a `.class` file provided that it can find the `.java` source file). The class files are written to the same directories as the corresponding source files.

To run the application using these class files, type the following command:

```
java -cp {your-path-up-to-and-including-the-source-  
directory};{path}antlr.jar com.jrefinery.report.demo.JFreeReportDemo
```

For example, while testing this on Windows NT 4.0, I entered the following:

```
java -cp F:\java\jfreereport\source;F:\java\jfreereport\jars\antlr.jar  
com.jrefinery.report.demo.JFreeReportDemo
```

Something similar should work on your system.

Generating the JavaDoc Documentation

The JavaDoc HTML files are included, ready-made, in the download. But if you make changes to **JFreeReport** and update the doc-comments, then you will want to regenerate the HTML files using the **javadoc** tool. Enter the following command:

```
javadoc -sourcepath {your-path-up-to-and-including-the-source-  
directory} -d {your-output-directory} com.jrefinery.report  
com.jrefinery.report.demo
```

Once again, in testing on Windows, I entered the following:

```
javadoc -sourcepath F:\java\jfreereport\source -d javadoc2  
com.jrefinery.report com.jrefinery.report.JFreeReportDemo
```

...where `javadoc2` is a newly created subfolder `F:\java\jfreereport\source\javadoc2` to avoid overwriting the existing documentation (in my test).

Class Reference

This section contains special notes and reference information for all the **JFreeReport** classes. This section is incomplete – further information will be provided in the next release of this document.

There are currently three packages:

- `com.jrefinery.report` – the main report classes;
- `com.jrefinery.report.io` – ANTLR-generated files for reading report definitions from text files;
- `com.jrefinery.report.demo` – the demonstration application;

Note that you only require the first two packages to use the **JFreeReport** class library in your own applications (along with `com.jrefinery.util.io` and `com.jrefinery.util.ui`).

Additional information can be found in the Javadoc-generated HTML documentation for **JFreeReport**.

com.jrefinery.report

This package contains most of the classes for the **JFreeReport** class library.

Band

A report band is a section of a report containing elements (display items). There are seven different band types:

- the report header;
- the report footer;
- the page header;
- the page footer;
- a group header;
- a group footer;
- one row of data items;

► `public void populateElements(TableModel data, int row);`

Before a band is displayed, the data values from the current row of the `TableModel` must be assigned to the data elements. The **JFreeReport** class will call this method at the appropriate time.

► `public void draw(Graphics2D g2, float x, float y);`

When the report is being previewed or printed, individual bands are drawn, using calls to the Java2D API, by this method.

See also: **ReportHeader, ReportFooter, PageHeader, PageFooter, GroupHeader, GroupFooter, ItemBand;**

Cursor

A utility class for keeping track of the current drawing position when rendering a report on a graphics device.

DataElement

An extension of **TextElement**, this class is the base class for all report elements that display data from the report's data source.

► **public void** set (Object value);

This method is used to populate the data element prior to its band being drawn

See also: **Element**;

DateElement

This class extends **DataElement**, and is used to display `java.util.Date` items from the report's data source.

► **protected** String getFormattedText ();

Returns a formatted version of the element's data value. Overridden to return a string representing the date formatted according to the default locale;

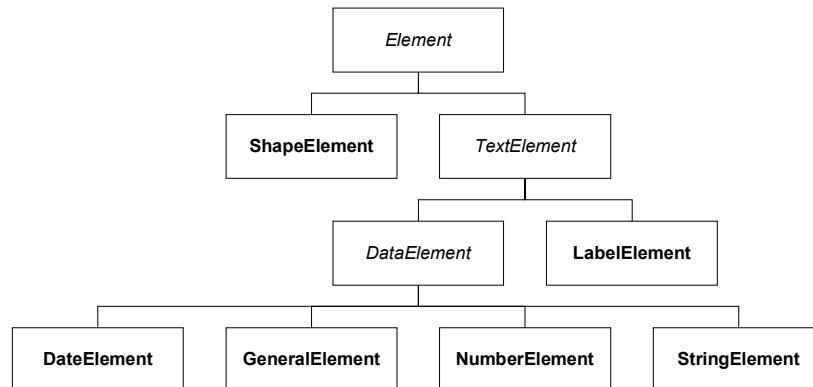
See also: **Element**, **DataElement**, **TextElement**;

Element

An *element* is any item that can be displayed within a report band. Elements can be used to display report data, or fixed labels (static text), or graphical shapes.

► Every element has a name that uniquely identifies the element within the report. This name is not used in the current implementation but is likely to be used in the future (for example, in the implementation of a GUI report designer);

► The following diagram shows the hierarchy of elements used in **JFreeReport**:



See also: **Band**;

GeneralElement

This class extends **DataElement**, and is used to display `java.lang.Objects` and any subclasses that are not Dates, Numbers or Strings.

See also: **Element**, **DataElement**, **TextElement**;

Group

This class defines a *group* for the report. A group is defined by a set of fields, and the items in the group are all the consecutive rows of data where the values in the group's fields do not change.

► A group has an optional *group header*, a report band that will be displayed every time a new group is started;

► A group has an optional *group footer*, a report band that will be displayed every time a group is finished;

► `public boolean lastItemInGroup(TableModel data, int itemIndex)`

Returns true if the current item is the last in this group, and false otherwise. This is determined by examining the values in each of the group's fields – if any of them changes between the current item and the next item, then we have reached the end of the current group:

See also: **GroupHeader, GroupFooter;**

GroupFooter

A *group footer* is a report band that appears at the end of every group in a report.

► A group footer, like all report bands, can contain any number of elements;

See also: **Group, GroupHeader;**

GroupHeader

A *group header* is a report band that appears at the end of every group in a report.

► A group header, like all report bands, can contain any number of elements;

See also: **Group, GroupFooter;**

ItemBand

A report band that appears once for each row of data in the report's `TableModel`.

► An item band, like all report bands, can contain any number of elements;

See also: **Band;**

JFreeReport

This class is the main co-ordinator of report printing. It maintains a state-model that keeps track of where the process is up to (so that printing can resume following a page break).

► **JFreeReport** gets data from any class that implements the `javax.swing.table.TableModel` interface. There are nine methods in the **TableModel** interface, but **JFreeReport** uses just two of them:

```
public String getColumnName(int columnIndex);
```

Subclasses of `DataElement` have a `Field` attribute – this should match the column name from which the data value is to be obtained;

```
public Object getValueAt(int row, int column);
```

This method is called to get an actual data value, once the column has been determined from the `Field` name;

► **JFreeReport** reads report definitions from a ZIP file. Any number of report definitions can be included in the ZIP file. The file should be stored on the classpath;

► **JFreeReport** uses a state model to keep track of progress with the report generation. A report has one or more groups, with the lowest group being the item group. There is a method to advance the state of the item group:

```
public boolean advanceItemGroup(ReportState rs, Graphics2D g2,  
    PageFormat pf, boolean draw)
```

Similarly, if there are higher level groups, and we are in the middle of processing one of them, we can use the following method:

```
public boolean advanceParentGroup(ReportState rs, Graphics2D g2,  
    PageFormat pf, boolean draw)
```

► One of the difficult issues with report generation is handling the various paper sizes to which the end-user may send the report. For simplicity, I've assumed that all reports are one page wide. At design time, the report designer designs the report with a particular paper-size in mind. At run-time, we don't change this, except that if the user selects a different paper size, we print what we can.

See also: **ReportHeader, ReportFooter, PageHeader, PageFooter;**

LabelElement

A label element displays static text within a report band. This class extends **TextElement**.

See also: **Element, TextElement;**

NumberElement

A *number element* is a data element that handles the display of data values that are subclasses of `java.lang.Number`;

► The constructors allow the number format to be specified as a `String` – this string is used by an instance of `java.text.NumberFormat` to format the number prior to displaying or printing it;

See also: **Element, DataElement, TextElement;**

PageFooter

A *page footer* is a report band that appears at the end of every page in a report.

► Page footers are optional – a report is not required to have a page footer.

See also: **Band, PageHeader;**

PageHeader

A *page header* is a report band that appears at the beginning of every page in a report.

► Page headers are optional – a report is not required to have a page header.

See also: **Band, PageFooter;**

PreviewFrame

A *preview frame* is a ready-made, stand-alone `JFrame` that can be used to preview a report.

► The current implementation is fairly basic, but contains the basic mechanism for paging forward and backward through the report. It also allows the user to “zoom in” and “zoom out” while viewing the report;

See also: **Band**, **ReportHeader**;

ReportFooter

A *report footer* is a report band that appears once-only at the end of a report.

- Report footers are optional – a report is not required to have a footer.

See also: **Band**, **ReportHeader**;

ReportHeader

A *report header* is a report band that appears once-only at the beginning of a report.

- Report headers are optional – a report is not required to have a header.

See also: **Band**, **ReportFooter**;

ReportPane

A *report pane* is a component used within the preview frame. You can use it if you want to implement your own print preview frame.

See also: **PreviewFrame**;

ReportState

This class stores state information for a report or report group. The **ReportState** contains a state record for each open group in the report.

{ insert a state diagram here. }

ShapeElement

A *shape element* is used for drawing shapes (typically lines and boxes) on a report.

- This class has not been finished in the current implementation. One problem to solve is how to allow lines to fill the width of the report, allowing for different paper sizes (that is, we require dynamic resizing of the shape);
-

StringElement

This class extends **DataElement**, and handles all data items that are `String` objects.

- In practice, this class behaves in exactly the same way as **GeneralElement**;

See also: **Element**, **DataElement**, **TextElement**;

TextElement

This class extends **Element**, and is the base class for all elements that display text.

See also: **DateElement**, **NumberElement**, **StringElement**;

com.jrefinery.report.io

This package contains three classes that are automatically generated using ANTLR. The ANTLR specification file appears in Appendix A.

ReportDefinitionParser

A parser for reading report definition files.

- The following text is an example of a report definition that can be parsed by this class:

```
// This is a sample report for testing purposes...

"Sample Report 1" (468, 648) {

    Report Header (24) {
        Label : "Label 1" (0, 0, 468, 24, ["Serif", BOLD, 16], CENTER, 20, "LIST OF COUNTRIES BY CONTINENT")
    }

    Report Footer (24) {
        Label : "Label 2" (0, 0, 468, 24, ["SansSerif", BOLD, 16], CENTER, 20, "**** END OF REPORT ****")
    }

    Page Header (18) {
        Label : "Label 3" (0, 0, 468, 18, ["Courier New", BOLD, 12], RIGHT, 15, "List of Countries by
Continent")
    }

    Page Footer (18) {
        Label : "Label 4" (0, 0, 468, 18, ["Courier New", BOLD, 12], RIGHT, 15, "Page x of y (not
implemented)")
    }

    Group Definitions {
        "Continent Group" ("Continent") {
            Group Header (18) {
                Label : "Label 5" (0, 0, 76, 18, ["Monospaced", BOLD, 10], LEFT, 14, "CONTINENT:")
                String : "Continent Element" (96, 0, 76, 18, ["Monospaced", BOLD, 10], LEFT, 14, "Continent")
            }
            Group Footer (18) {
                Label : "Label 6" (0, 0, 468, 18, ["Monospaced", BOLD, 10], RIGHT, 14, "END OF CONTINENT GROUP")
            }
        }
    }

    Items (12) {
        String : "Country Element" (0, 0, 76, 12, ["Monospaced", PLAIN, 9], LEFT, 10, "Country")
        String : "Code Element" (180, 0, 76, 12, ["Monospaced", PLAIN, 9], LEFT, 10, "ISO Code")
        Number : "Population Element" (240, 0, 76, 12, ["Monospaced", PLAIN, 9], RIGHT, 10, "Population",
"###0")
    }
}
```

- This class is automatically generated using ANTLR;
-

ReportDefinitionParserTokenTypes

This class defines the tokens generated by the lexer and consumed by the parser.

- This class is automatically generated using ANTLR;
-

ReportLexer

A lexical analyser for the report definitions.

- This class is automatically generated using ANTLR;

APPENDIX A – Report Definition Parser Specification

I've used ANTLR to generate a parser that reads report definitions from a text file. For more information on the ANTLR tool, please refer to the official ANTLR web-site:

<http://www.antlr.org>

The parser specification – an input file for the ANTLR tool – is listed below:

```
// ANTLR specifications for a parser for a report definition.
//
// Note that some of the productions in the grammar are repetitious. They have been
// left this way as it is easier to follow what is going on.

// *** PARSE ***
header { package com.jrefinery.report.io; }

{import java.awt.*;
import java.awt.geom.*;
import java.util.*;
import com.jrefinery.report.*; }

class ReportDefinitionParser extends Parser;

options { k=6; defaultErrorHandler=false; }

{ JFreeReport report; }

getReportDefinition returns [JFreeReport report=null] { Dimension size=null; }
: name:STRING size=readDefaultSize LEFTBRACE report=readDefinition RIGHTBRACE
(SEMICOLON)?
;

readDefaultSize returns [Dimension d=null] { int w=0; int h=0; }
: LEFTBRACKET w=readInt COMMA h=readInt RIGHTBRACKET { d=new Dimension(w, h); }
;

readDefinition returns [JFreeReport report=null] { ReportHeader rh=null; ReportFooter
rf=null;
PageHeader ph=null; PageFooter
pf=null;
java.util.List groups=null;
ItemBand items=null; }
: (REPORT HEADER rh=readReportHeader)?
(REPORT FOOTER rf=readReportFooter)?
(PAGE HEADER ph=readPageHeader)?
(PAGE FOOTER pf=readPageFooter)?
(GROUP DEFINITIONS groups=readGroups)?
ITEMS items=readItems
{ report = new JFreeReport(rh, ph, items, pf, rf, groups, null, null); }
;

readReportHeader returns [ReportHeader header=null] { int h=0; Collection
elements=null; }
: LEFTBRACKET h=readInt RIGHTBRACKET elements=listOfElements
{ header = new ReportHeader(new Integer(h), elements); }
(SEMICOLON)?
;

readReportFooter returns [ReportFooter footer=null] { int h=0; Collection
elements=null; }
: LEFTBRACKET h=readInt RIGHTBRACKET elements=listOfElements
{ footer = new ReportFooter(new Integer(h), elements); }
(SEMICOLON)?
;

readPageHeader returns [PageHeader header=null] { int h=0; Collection elements=null; }
: LEFTBRACKET h=readInt RIGHTBRACKET elements=listOfElements
{ header = new PageHeader(new Integer(h), elements); }
(SEMICOLON)?
;
```

```

readPageFooter returns [PageFooter footer=null] { int h=0; Collection elements=null; }
: LEFTBRACKET h=readInt RIGHTBRACKET elements=listOfElements
  { footer = new PageFooter(new Integer(h), elements); }
  (SEMICOLON)?
;

readGroups returns [java.util.List groups=null] { groups = new ArrayList(); }
: LEFTBRACE groups = readGroupList RIGHTBRACE
;

readGroupList returns [java.util.List groups=null] { Group group=null; groups = new
ArrayList(); }
: (group = readGroup { groups.add(group); } ) *
;

readGroup returns [Group group=null] { java.util.List fields=null; GroupHeader
gh=null; GroupFooter gf=null; }
: name:STRING LEFTBRACKET fields=readFields RIGHTBRACKET
  LEFTBRACE (GROUP HEADER gh=readGroupHeader)? (GROUP FOOTER
gf=readGroupFooter)? RIGHTBRACE (SEMICOLON)?
  { group=new Group(name.getText(), gh, gf, fields); }
;

readFields returns [java.util.List fields=null] { fields = new ArrayList(); }
: (name:STRING { fields.add(name.getText()); } ) ?
  (COMMA name2:STRING { fields.add(name2.getText()); } ) *
;

readGroupHeader returns [GroupHeader header=null] { int h=0; Collection elements=null;
}
: LEFTBRACKET h=readInt RIGHTBRACKET elements=listOfElements
  { header = new GroupHeader(new Integer(h), elements); }
  (SEMICOLON)?
;

readGroupFooter returns [GroupFooter footer=null] { int h=0; Collection elements=null;
}
: LEFTBRACKET h=readInt RIGHTBRACKET elements=listOfElements
  { footer = new GroupFooter(new Integer(h), elements); }
  (SEMICOLON)?
;

readItems returns [ItemBand items=null] { Collection elements=null; int height=0; }
: LEFTBRACKET height=readInt RIGHTBRACKET elements=listOfElements
  { items = new ItemBand(new Integer(height), elements); }
  (SEMICOLON)?
;

listOfElements returns [Collection elements=null] { elements = new ArrayList();
Element element=null; }
: LEFTBRACE (element=readElement { elements.add(element); } ) * RIGHTBRACE
;

readElement returns [Element element=null]
: LABEL_ELEMENT COLON element=readLabelElement
| SHAPE_ELEMENT COLON element=readShapeElement
| GENERAL_ELEMENT COLON element=readGeneralElement
| STRING_ELEMENT COLON element=readStringElement
| NUMBER_ELEMENT COLON element=readNumberElement
| DATE_ELEMENT COLON element=readDateElement
;

readLabelElement returns [LabelElement element=null]
: { int x=0; int y=0; int w=0; int h=0; Font f = null; int align =
Element.LEFT; int baseline; }
  name:STRING LEFTBRACKET
  x=readInt COMMA y=readInt COMMA w=readInt COMMA h=readInt COMMA
  f=readFont COMMA align=readAlign COMMA baseline = readInt COMMA text:STRING
RIGHTBRACKET
  { element = new LabelElement(name.getText(), x, y, w, h, f, align, baseline,
text.getText()); }
  (SEMICOLON)?
;

readShapeElement returns [ShapeElement element=null]
: { int x=0; int y=0; int w=0; int h=0; }
  name:STRING LEFTBRACKET

```

```

        x=readInt COMMA y=readInt COMMA w=readInt COMMA h=readInt COMMA
RIGHTBRACKET
        { element = new ShapeElement("No name", new Rectangle2D.Float(0, 0, 10,
10)); }
        (SEMICOLON)?
    ;

readGeneralElement returns [GeneralElement element=null]
    : { int x=0; int y=0; int w=0; int h=0; Font f = null; int align =
Element.LEFT; int baseline=0; }
    name:STRING LEFTBRACKET
    x=readInt COMMA y=readInt COMMA w=readInt COMMA h=readInt COMMA
    f=readFont COMMA align=readAlign COMMA baseline=readInt COMMA field:STRING
RIGHTBRACKET
    { element = new GeneralElement(name.getText(), x, y, w, h, f, align,
baseline, field.getText()); }
    (SEMICOLON)?
    ;

readStringElement returns [StringElement element=null]
    : { int x=0; int y=0; int w=0; int h=0; Font f = null; int align =
Element.LEFT; int baseline=0; }
    name:STRING LEFTBRACKET
    x=readInt COMMA y=readInt COMMA w=readInt COMMA h=readInt COMMA
    f=readFont COMMA align=readAlign COMMA baseline=readInt COMMA field:STRING
RIGHTBRACKET
    { element = new StringElement(name.getText(), x, y, w, h, f, align,
baseline, field.getText()); }
    (SEMICOLON)?
    ;

readNumberElement returns [NumberElement element=null]
    : { int x=0; int y=0; int w=0; int h=0; Font f = null; int align =
Element.LEFT; int baseline=0; }
    name:STRING LEFTBRACKET
    x=readInt COMMA y=readInt COMMA w=readInt COMMA h=readInt COMMA
    f=readFont COMMA align=readAlign COMMA baseline=readInt COMMA field:STRING
COMMA format:STRING RIGHTBRACKET
    { element = new NumberElement(name.getText(), x, y, w, h, f, align,
baseline, field.getText(), format.getText()); }
    (SEMICOLON)?
    ;

readDateElement returns [DateElement element=null]
    : { int x=0; int y=0; int w=0; int h=0; Font f = null; int align =
Element.LEFT; int baseline=0; }
    name:STRING LEFTBRACKET
    x=readInt COMMA y=readInt COMMA w=readInt COMMA h=readInt COMMA
    f=readFont COMMA align=readAlign COMMA baseline=readInt COMMA field:STRING
COMMA format: STRING RIGHTBRACKET
    { element = new DateElement(name.getText(), x, y, w, h, f, align, baseline,
field.getText(), format.getText()); }
    (SEMICOLON)?
    ;

readInt returns [int result]
    : token:INT { result = Integer.parseInt(token.getText()); }
    ;

readFont returns [Font font=null] { int style; }
    : LEFTSQUAREBRACKET name:STRING COMMA style=readStyle COMMA size:INT
RIGHTSQUAREBRACKET
    { font = new Font(name.getText(), style, Integer.parseInt(size.getText())); }
    ;

readStyle returns [int style]
    : FONT_PLAIN { style = Font.PLAIN; }
    | FONT_BOLD { style = Font.BOLD; }
    | FONT_ITALIC { style = Font.ITALIC; }
    | FONT_BOLDITALIC { style = Font.BOLD+Font.ITALIC; }
    ;

readAlign returns [int align]
    : ALIGN_LEFT { align = 1; }
    | ALIGN_RIGHT { align = 2; }
    | ALIGN_CENTER { align = 3; }
    ;

```

```

;

// *** LEXICAL ANALYSER ***
class ReportLexer extends Lexer;

options { k=3; charVocabulary = '\3'..'177'; }

COMMENT      : "//" (~'\n')* '\n'! { $setType(Token.SKIP); newline(); }
;

STRING       : "'"! (~'"')* "'"!
;

protected
DIGIT : '0'..'9'
;
INT : (DIGIT)+
;
REPORT      : "Report" | "REPORT" | "report";
PAGE        : "Page" | "PAGE" | "page";
GROUP       : "Group" | "GROUP" | "group";
HEADER      : "Header" | "HEADER" | "header";
FOOTER      : "Footer" | "FOOTER" | "footer";
DEFINITIONS : "Definitions" | "DEFINITIONS" | "definitions";
ITEMS       : "Items" | "ITEMS" | "items";

LABEL_ELEMENT : "Label";
SHAPE_ELEMENT : "Shape";
GENERAL_ELEMENT : "General";
STRING_ELEMENT : "String";
NUMBER_ELEMENT : "Number";
DATE_ELEMENT  : "Date";

FONT_PLAIN    : "PLAIN";
FONT_BOLD    : "BOLD";
FONT_ITALIC   : "ITALIC";
FONT_BOLDITALIC : "BI";

ALIGN_LEFT    : "LEFT";
ALIGN_RIGHT   : "RIGHT";
ALIGN_CENTER  : "CENTER" | "CENTRE";

IDENTIFIER : ('a'..'z'|'A'..'Z')+
;

DASH : '-'
;
EQUALS : '='
;
COMMA : ','
;
LEFTBRACE : '{'
;
RIGHTBRACE : '}'
;
LEFTBRACKET : '('
;
RIGHTBRACKET : ')'
;
LEFTSQUAREBRACKET : '['
;
RIGHTSQUAREBRACKET : ']'
;
COLON : ':'
;
SEMICOLON : ';'
;
// ignore white space
WS : (' ' | '\t' | '\n' { newline(); } | '\r') { $setType(Token.SKIP); }
;

```

APPENDIX B – The GNU Lesser General Public Licence

JFreeReport is licensed under the terms of the GNU Lesser General Public Licence (GPL), a copy of which appears below.

The following text is reproduced from the web-site of the Free Software Foundation (<http://www.fsf.org>):

GNU LESSER GENERAL PUBLIC LICENSE

Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc.
59 Temple Place, Suite 330, Boston, MA 02111-1307 USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts
as the successor of the GNU Library Public License, version 2, hence
the version number 2.1.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users.

This license, the Lesser General Public License, applies to some specially designated software packages--typically libraries--of the Free Software Foundation and other authors who decide to use it. You can use it too, but we suggest you first think carefully about whether this license or the ordinary General Public License is the better strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish); that you receive source code or can get it if you want it; that you can change the software and use pieces of it in new free programs; and that you are informed that you can do these things.

To protect your rights, we need to make restrictions that forbid distributors to deny you these rights or to ask you to surrender these rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link other code with the library, you must provide complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this license, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others.

Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive license from a patent holder. Therefore, we insist that any patent license obtained for a version of the library must be consistent with the full freedom of use specified in this license.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License. This license, the GNU Lesser General Public License, applies to certain designated libraries, and is quite different from the ordinary General Public

License. We use this license for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public License therefore permits such linking only if the entire combination fits its criteria of freedom. The Lesser General Public License permits more lax criteria for linking other code with the library.

We call this license the "Lesser" General Public License because it does Less to protect the user's freedom than the ordinary General Public License. It also provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public License for many libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public License.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public License is Less protective of the users' freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, whereas the latter must be combined with the library in order to run.

TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- * a) The modified work must itself be a software library.
- * b) You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
- * c) You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
- * d) If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law. If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

- * a) Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)
- * b) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.
- * c) Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.
- * d) If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.
- * e) Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

- * a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

* b) Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS